VIREL V3 — Проектная документация

Версия: 1.0
Дата: 14 марта 2026
Проект: Virel — Operations Platform v3
Разработчик: Adel Islamov
Репозиторий: github.com/adelislamov-star/virel-v3
Production: https://virel-v3.vercel.app

---

1. ОБЩАЯ ИНФОРМАЦИЯ О ПРОЕКТЕ

Назначение системы

Virel v3 — операционная платформа для управления модельным агентством в Лондоне. Система автоматизирует полный цикл работы агентства: от добавления новых моделей и управления их профилями до приёма заявок на бронирование, обработки платежей и аналитики.

Система решает следующие задачи:

- Управление базой моделей: добавление анкет, хранение профилей, контроль статусов
- Автоматизация рабочего процесса рецепции: приём и подтверждение заявок на бронирование
- Публичное представление агентства: сайт для клиентов с каталогом, формой бронирования и SEO-оптимизацией
- Аналитика и CRM: отслеживание просмотров, предпочтений клиентов, выручки
- Интеграция с AI: автозаполнение анкет, генерация bio, подбор моделей, проверка фото

Технологический стек

Компонент	Технология
Frontend / Backend	Next.js 14 (App Router), TypeScript
База данных	PostgreSQL (Neon, EU Central)
ORM	Prisma
Хостинг	Vercel (Production + Preview)
Хранение файлов	Cloudflare R2 (EU bucket: virel-operations-eu)
CDN для фото	https://pub-2df895cf47a14d359a7341e0cda3efaf.r2.dev

Компонент	Технология
AI	Anthropic Claude (claude-sonnet-4-20250514)
Email	Resend (инфраструктура готова, токены не подключены)
Telegram	Bot API (инфраструктура готова, токены не подключены)
Мониторинг	Sentry
Rate Limiting	Upstash Redis
Стилизация	Tailwind CSS
Аутентификация	Кастомная система (JWT cookie, UserRole junction table)

2. АРХИТЕКТУРА СИСТЕМЫ

Общая структура

Система состоит из двух основных частей, работающих на одном Next.js приложении:

```
virel-v3.vercel.app/
├── /admin          — Back-office (DAC-office)
└── /               — Front-office (публичный сайт)
```

Обе части работают с единой базой данных PostgreSQL через Prisma ORM. Разграничение доступа реализовано через RBAC с 4 ролями: VIEWER, OPERATOR, OPS_MANAGER, OWNER.

2.1 DAC-office (Back-office)

Back-office расположен по адресу `/admin` и доступен только авторизованным пользователям. Предназначен для команды агентства: рецепции, менеджмента, владельца.

Разделы back-office:

Раздел	Описание
Dashboard	Сводная панель: выручка, операции, модели, Reception Overview
Booking Requests	Рабочий стол рецепции: входящие заявки, подтверждение, история
Bookings	Подтверждённые бронирования
Models	Управление анкетами моделей: CRUD, Quick Upload, AI Tools
Clients	CRM клиентов: история визитов, предпочтения, расходы
Masters / Services	Справочник сервисов агентства
Masters / Call Rates	Справочник длительностей и тарифных шаблонов
Masters / Locations	Справочник районов Лондона и станций метро
Blog	Управление публикациями
Concierge	Управление партнёрами (рестораны, отели, транспорт)
Payments	Ручной учёт платежей с CSV-экспортом
Analytics	Аналитика: Model Performance, Owner Analytics, Unit Economics
Settings	Конфигурация системы, статус интеграций
Audit Log	Журнал всех действий пользователей

2.2 Front-office (Публичный сайт)

Публичный сайт расположен в корне приложения и доступен без авторизации. Предназначен для клиентов агентства.

Страницы front-office:

Страница	URL	Описание
Главная	/	Hero, Featured Companions, Districts, Services, Smart Match
Каталог	/companions	Фильтруемый список моделей
Профиль модели	/companions/[slug]	Полная страница с Gallery, Booking Widget, Stats, Services
Бронирование	/book	3-шаговая форма создания заявки
Smart Match	/find-your-match	AI-квиз для подбора модели
Сервисы	/services	Каталог публичных сервисов
Сервис	/services/[slug]	Страница отдельного сервиса с моделями
Район	/london/[slug]-escorts/	SEO-страница района с моделями
Станция	/london/[slug]/[station]-station/	SEO-страница станции метро
О нас	/about	Информация об агентстве
Контакты	/contact	Контактная информация
FAQ	/faq	Часто задаваемые вопросы
Блог	/blog, /blog/[slug]	Публикации агентства
Консьерж	/concierge	Партнёрские сервисы

2.3 Модуль моделей

Центральный модуль системы. Модель (Model) — основная сущность, описывающая компаньона агентства.

Схема данных модели включает:

- Базовая информация: имя, slug, статус, публичный код
- Физические параметры: рост, вес, параметры фигуры, цвет волос/глаз, татуировки
- Личные данные: возраст, национальность, этническая принадлежность, языки, ориентация
- Маркетинг: tagline, bio, availability, publicTags, isExclusive, isVerified
- Контакты (внутренние): телефон, email, мессенджеры
- Локация: ближайшая станция, почтовый индекс, адрес

- Финансы: тарифы (ModelRate), платёжные методы
- Сервисы: привязанные сервисы (ModelService), wardrobe
- Аналитика: viewsTotal, viewsToday, lastViewedAt
- Медиа: coverPhotoUrl, galleryPhotoUrls

Статусы модели: Active / Vacation / Archived

2.4 Модуль загрузки анкет (Quick Upload)

Quick Upload — механизм добавления новых моделей через AI-обработку анкеты.

Расположение в системе:

- `/admin/models` → кнопка "Quick Upload"
- `/admin/models/new` → Шаг 4 (Photos) → блок загрузки анкеты

Принцип работы:

1. Пользователь загружает PDF или фото анкеты
2. Файл передаётся в Anthropic Claude с детальным prompt
3. Claude извлекает 35+ полей из документа
4. Система создаёт запись Model в БД
5. Создаются связанные записи: ModelRate, ModelService
6. Фото загружаются в Cloudflare R2 с водяным знаком
7. Пользователь перенаправляется на созданный профиль с отчётом

2.5 Модуль AI (Artificial Intelligence)

Система использует Anthropic Claude для следующих функций:

Функция	Endpoint	Описание
Quick Upload	POST /api/v1/models/quick-upload	Извлечение данных из анкеты
Generate Bio	POST /api/v1/models/[id]/generate-bio	Генерация биографии модели
Verify Photos	POST /api/v1/models/[id]/verify-photos	Проверка подлинности фото (vision)
Generate SEO	POST /api/v1/models/[id]/generate-seo	Генерация SEO title и description
Smart Match	POST /api/public/smart-match	Подбор моделей по предпочтениям клиента
Service SEO	POST /api/v1/services/[id]/generate-seo	SEO для страниц сервисов
District SEO	POST /api/v1/districts/[id]/generate-seo	SEO для страниц районов
Blog Generate	POST /api/v1/blog/[id]/generate	Генерация контента для блога

Все AI-эндпоинты реализуют graceful degradation: при отсутствии `ANTHROPIC_API_KEY` возвращают 503 с понятным сообщением.

2.6 Модуль бронирования

Реализует полный цикл обработки заявки на бронирование.

Сущности: BookingRequest (заявка), Booking (подтверждённое бронирование), Client (клиент).

Статусы заявки: pending → confirmed / cancelled / rejected

2.7 Система уведомлений

Инфраструктура уведомлений полностью реализована. Токены не подключены — система работает в режиме логирования.

Email (Resend): 4 шаблона — booking_received, booking_confirmed, booking_cancelled, booking_reminder. Стиль: тёмный фон `#0a0a0a`, gold акценты `#c9a96e`, шрифт Georgia serif.

Telegram: функции notifyReception() и notifyTommy(). 5 шаблонов сообщений: новая заявка, подтверждение, 30-минутный напоминатель, 2-часовой напоминатель, критический алерт.

2.8 Cron Jobs

14 автоматических задач в `vercel.json`, защищённых `CRON_SECRET`:

Job	Расписание	Назначение
notification-dispatch	*/5 * * * *	Обработка очереди уведомлений
fraud-scan	*/15 * * * *	Автоматическое сканирование на мошенничество
booking-reminder	*/15 * * * *	Email клиентам за 2 часа до встречи
booking-reminder-30	*/15 * * * *	Telegram рецепции за 30 минут
operations-feed	0 * * * *	Обновление операционной ленты
lost-revenue	0 * * * *	Расчёт потерянной выручки
demand-stats	0 */6 * * *	Статистика спроса
retention-scan	0 2 * * *	Анализ удержания клиентов
staff-snapshots	0 3 * * *	Снимки данных по сотрудникам
analytics-rebuild	0 4 * * *	Пересчёт аналитических данных
reset-daily-views	0 0 * * *	Сброс счётчика viewsToday

---

3. РЕАЛИЗОВАННЫЕ ИЗМЕНЕНИЯ ЗА ТЕКУЩУЮ РАБОЧУЮ СЕССИЮ

3.1 Расширение схемы базы данных

В модель `Model` добавлено более 40 новых полей:

- **Контактные данные:** phone, phone2, email, whatsapp, telegram, viber, signal
- **Физические параметры:** hairLength, dressSize, footSize, ethnicity
- **Адресные данные:** postcode, street, houseNumber, floor, flatNumber, nearestStation
- **Предпочтения:** worksWithCouples, worksWithWomen, willingDisabled, willingPrivatePlaces, willingInternational, willingLongDistance, hasFlatmates, dinnerDates, travelCompanion, offersIncall, offersOutcall
- **Аналитика:** viewsTotal, viewsToday, lastViewedAt
- **Платёжные методы:** paymentCash, paymentTerminal, paymentBankTransfer, paymentMonese, paymentMonzo, paymentRevolut, paymentStarling, paymentBTC, paymentLTC, paymentUSDT

Добавлены новые модели в схему: UserRole (junction table для RBAC), структуры для аналитики.

3.2 Файл констант модельной формы

Создан файл `src/constants/model-form.ts` со всеми справочными данными рынка:

- 131 станция метро Лондона
- 76 национальностей
- 39 языков с уровнями владения
- Полный список bust sizes (28AA-42D)
- Heights (150-185cm) с конвертацией в футы
- Weights (40-89kg) с конвертацией в фунты
- Dress sizes, Foot sizes, Wardrobe (27 опций)

3.3 4-шаговый Wizard для создания модели

Переработана страница `/admin/models/new`. Вместо одной длинной формы — пошаговый wizard:

- **Шаг 1:** Personal Information (физические параметры, языки, предпочтения)
- **Шаг 2:** Rates & Services (тарифы по длительностям, сервисы по категориям, wardrobe, платёжные методы)
- **Шаг 3:** Location & Availability (контакты внутренние, адрес incall, расписание)
- **Шаг 4:** Photos & Save (загрузка фото, загрузка анкеты для AI, финальное сохранение)

3.4 Новые секции на странице редактирования модели

На странице `/admin/models/[id]` добавлены секции:

- **Contact:** телефоны, email, мессенджеры (whatsapp, telegram, viber, signal)
- **Payment Methods:** 10 методов оплаты с чекбоксами

3.5 Исправление Services API и UI

Исправлено расхождение структуры данных:

- API `/api/v1/services` переведён с группированного формата `{ data: { categories } }` на плоский массив `{ services, stats }`
- UI обновлён для работы с новой структурой
- Категории в dropdown исправлены: старые значения (Connection, Oral, Group) заменены на актуальные (signature, intimate, wellness, fetish, bespoke)
- Добавлена колонка Models с подсчётом привязанных моделей
- Stat cards (Total/Public/Members Only/Active) получают данные из API вместо локального подсчёта

3.6 Исправление Blog авторизации

Снята избыточная проверка роли в GET `/api/v1/blog`. Страница `/admin/blog` перестала падать с Application Error для авторизованных пользователей с недостаточной ролью.

3.7 Публичный фронт — главная страница

Обновлена главная страница:

- Hero section подключён к реальным данным (coverPhotoUrl первой featured модели)
- Секция Featured Companions: карточки ModelCard с реальными данными из БД
- Секции Districts и Services подключены к БД
- Добавлен блок Smart Match
- ISR revalidate = 3600

3.8 Публичный фронт — каталог компаньонов

Создана/обновлена страница `/companions`:

- Фильтрация через URL params (Server Component): по цвету волос, национальности, району, доступности, возрасту, сервису, ценовому диапазону
- Сортировка: recommended, newest, price-asc, price-desc
- Пагинация по 20 записей с SEO-friendly ссылками
- Пустое состояние при отсутствии результатов
- Sidebar фильтры как Client Component с router.push

3.9 Публичный фронт — профиль модели

Создана страница `/companions/[slug]`:

- Gallery с thumbnails (Client Component)
- Booking Widget (sticky, Client Component): таблица цен incall/outcall, кнопки Book Now/WhatsApp/Telegram
- Блок статистики: показываются только заполненные поля
- Блок Location: районы и nearestStation
- Блок Services: только публичные (category != intimate), разделение regular/extras
- Блок Wardrobe: если не пустой
- Блок DUO Partners: если есть duoPartnerIds
- generateStaticParams, generateMetadata, Schema.org Person, notFound()

3.10 Форма бронирования /book

Создана 3-шаговая форма:

- Шаг 1: выбор модели, тип (incall/outcall), дата/время, длительность с ценами, location, extras, occasion
- Шаг 2: контактные данные клиента
- Шаг 3: подтверждение с резюме, disclaimer, обязательный age verification checkbox
- Real-time калькулятор: base + extras = total
- Pre-fill из URL: `/book?modelSlug=sofia`
- Success screen с reference номером после отправки

3.11 Smart Match /find-your-match

Создан квиз из 5 вопросов:

1. Тип опыта (GFE / PSE / Wellness / Fetish)
2. Внешность (блондинка / брюнетка / рыжая / тёмная / без предпочтений)
3. Район (выбор из списка или "flexible")
4. Длительность (1h / 2h / 3h / overnight)
5. Бюджет (до £400 / £400-700 / £700-1200 / £1200+)

Переход между вопросами — автоматический при выборе ответа (fade анимация). После последнего вопроса — запрос к `/api/public/smart-match`, отображение 2-4 результатов с объяснением от AI.

3.12 Страницы сервисов

Созданы страницы `/services` (список по категориям без intimate/bespoke) и `/services/[slug]` (модели с этим сервисом, intro/full text, related services, Schema.org Service).

3.13 Страницы районов Лондона

Созданы SEO-страницы `/london/[slug]-escorts/` (25 страниц) с:

- Моделями в районе
- Transport hubs с временем ходьбы
- Hotels и Restaurants из поля district.hotels[]/restaurants[]
- Соседними районами для internal linking
- FAQ блоком
- Schema.org LocalBusiness + BreadcrumbList

Созданы страницы `/london/[slug]/[station]-station/` (20 страниц). При description < 50 слов — robots: noindex.

3.14 Статичные страницы

Созданы страницы: `/about`, `/contact`, `/faq` (с Schema.org FAQPage), `/blog`, `/concierge`. Устранены 404 ошибки на этих URL.

3.15 Sitemap и Robots

Созданы `src/app/sitemap.ts` и `src/app/robots.ts`:

- Sitemap включает все динамические страницы (модели, сервисы, районы, хабы, посты) + статичные
- Robots: /admin/, /api/, /members/ в Disallow

3.16 Trust & Safety

- **Panic Button:** кнопка ✕ в header публичного сайта, клик → редирект на bbc.co.uk/news
- **Verification Badges:** "✓ Verified Photos", "✓ Genuine Photos", "● Active Today" в BookingWidget и на карточках моделей
- **Disclaimer:** в footer всех страниц, на профилях моделей, на /book перед Submit
- **Age Verification:** обязательный checkbox "I confirm I am 18 years or older" на шаге 3 /book. Submit disabled без подтверждения
- **GDPR Cookie Banner:** появляется при первом посещении, сохраняет выбор (Accept All / Essential Only) в localStorage

3.17 SEO Technical

- `generateMetadata()` добавлен на все динамические страницы: companions/[slug], services/[slug], london/[slug], london/[slug]/[station], blog/[slug]
- `metadata` добавлен на все статичные страницы
- `/book` получил `robots: noindex`
- Schema.org разметка на всех страницах: Person (профили), LocalBusiness (районы), Service (сервисы), FAQPage (faq), Article (blog), Organization+LocalBusiness (главная)
- ISR revalidate: 3600s (большинство страниц), 1800s (каталог), 86400s (статичные)
- `<img>` теги заменены на `next/image` везде
- R2 домен добавлен в `next.config.js` remotePatterns

3.18 Notification Infrastructure

Создана полная инфраструктура уведомлений без подключения токенов:

Email (`src/lib/email.ts`): функции sendBookingReceived, sendBookingConfirmed, sendBookingCancelled, sendBookingReminder. При отсутствии RESEND_API_KEY — логирование и return false (graceful degradation).

Telegram (`src/lib/telegram.ts`): функции sendTelegramMessage, notifyReception, notifyTommy. При отсутствии токенов — аналогичная graceful degradation.

Шаблоны сообщений (`src/lib/telegram-messages.ts`): newBookingRequestMessage, bookingConfirmedMessage, bookingReminder30Message, bookingReminder2hMessage, criticalAlertMessage.

Уведомления подключены в:

- `POST /api/public/booking-request` — email клиенту + Telegram рецепции
- `POST /api/v1/booking-requests/[id]/confirm` — email подтверждения + Telegram
- `PUT /api/v1/booking-requests/[id]` при отмене — email отмены

3.19 Analytics & CRM (Шаг 9)

View Tracking:

- Добавлены поля viewsTotal, viewsToday, lastViewedAt в модель Model
- Создан `POST /api/public/models/[slug]/view` — инкремент счётчиков
- Компонент `<ViewTracker>` вызывается при открытии профиля
- Cron `reset-daily-views` сбрасывает viewsToday в 00:00

Dashboard Reception Overview (4 новых виджета):

1. Pending Booking Requests — последние 5 pending заявок с кликом
2. Confirmations Needed — заявки < 24ч без подтверждения
3. Top Viewed Models Today — топ по viewsToday
4. Weekly Revenue vs Last Week — с процентным изменением

Client CRM обновления:

- Total Spend из подтверждённых бронирований
- Visit Count и Last Visit
- Preferred Models (топ-3 по частоте)
- Quick Re-Book кнопка с pre-fill
- Internal Notes с сохранением через PATCH /api/v1/clients/:id/notes

Models список:

- Карточки показывают Views / Bookings / Conversion Rate

3.20 Footer с SEO-ссылками

Создан/обновлён компонент Footer с 4 колонками:

- Companions: All Companions, Find Your Match, Book Now, Concierge
- Services: топ-8 популярных публичных сервисов из БД с ссылками
- London Areas: топ-8 Tier 1 районов из БД с ссылками на district pages
- Agency: About, Contact, FAQ, Blog

3.21 Исправление системы аутентификации

Устранена проблема входа после изменений схемы:

- Исправлено поле `password` → `passwordHash` в логике auth
- Исправлена работа с UserRole junction table (роли хранятся отдельно от User)
- Создан скрипт для создания admin пользователя через upsert с upsert UserRole

3.22 Водяной знак на фото

Разработано ТЗ и инфраструктура для наложения водяного знака:

- Библиотека Sharp для обработки изображений в Node.js
- Текст "VIREL", Georgia serif, внизу по центру, 35% прозрачности
- Размер шрифта: 7% от ширины изображения (пропорционально)
- Применяется при загрузке в R2 перед сохранением
- Graceful fallback: при ошибке Sharp — сохранение оригинала

---

4. НОВЫЕ ФУНКЦИИ И ВОЗМОЖНОСТИ

4.1 Quick Upload с AI-обработкой анкет

Назначение: автоматическое создание профиля модели из анкеты без ручного заполнения.

Принцип работы: пользователь загружает PDF или фотографию анкеты. Система передаёт документ в Claude с детальным prompt, содержащим полный список 35+ полей с допустимыми значениями для каждого. Claude возвращает JSON. Система надёжно парсит ответ, создаёт запись Model, формирует связанные ModelRate и ModelService, загружает фото в R2 с водяным знаком, возвращает отчёт о заполненных и незаполненных полях.

Где используется: `/admin/models` (кнопка Quick Upload), `/admin/models/new` (Шаг 4).

4.2 Smart Match

Назначение: помощь клиенту в выборе подходящей модели через интерактивный квиз.

Принцип работы: 5 последовательных вопросов с автоматическим переходом при выборе ответа. После последнего вопроса POST-запрос к `/api/public/smart-match`. Claude

анализирует ответы, выбирает 2-4 модели из БД и формирует персональное объяснение для каждой. Отображаются карточки моделей с кнопками View Profile и Book Now.

Где используется: `/find-your-match`.

4.3 Model Performance Tracking

Назначение: отслеживание популярности и эффективности каждой модели.

Принцип работы: при каждом открытии профиля модели Client Component `<ViewTracker>` отправляет POST-запрос к `/api/public/models/[slug]/view`. Endpoint инкрементирует viewsTotal и viewsToday. Ежедневно в 00:00 cron `reset-daily-views` сбрасывает viewsToday до 0. Dashboard показывает топ моделей по дневным просмотрам.

Где используется: профили моделей (публичный фронт), Dashboard (admin), список Models (admin).

4.4 Reception Overview Dashboard

Назначение: сводная панель для рецепции с оперативной информацией.

Принцип работы: 4 виджета загружаются параллельно из БД при открытии Dashboard. Данные актуальны на момент загрузки страницы. Каждый виджет имеет собственный API endpoint.

Где используется: `/admin/dashboard`.

4.5 Watermark System

Назначение: защита фотографий моделей от несанкционированного копирования.

Принцип работы: при загрузке фото в R2 библиотека Sharp накладывает SVG-слой с текстом "VIREL" (Georgia serif, белый, 35% прозрачности) в нижней части изображения по центру. Размер шрифта пропорционален ширине изображения. Фотография сохраняется с водяным знаком — оригинал без метки не хранится.

Где используется: все upload endpoints для фото моделей.

4.6 GDPR Cookie Banner

Назначение: соответствие требованиям GDPR для пользователей из Европы.

Принцип работы: при первом посещении сайта отображается баннер в нижней части экрана. Пользователь выбирает Essential Only или Accept All. Выбор сохраняется в localStorage по ключу `virel-cookie-consent`. При последующих визитах баннер не показывается.

Где используется: все публичные страницы через `src/app/(public)/layout.tsx`.

---

5. ЛОГИКА РАБОТЫ КЛЮЧЕВЫХ МОДУЛЕЙ

5.1 DAC-office — работа рецепции с заявками

1. Клиент заполняет форму на `/book` → POST `/api/public/booking-request`
2. Система создаёт BookingRequest в БД со статусом `pending`
3. Отправляется email клиенту (booking_received) и Telegram рецепции
4. Рецепционист видит заявку в `/admin/booking-requests` (auto-refresh 30 сек)
5. Заявки > 30 минут без ответа подсвечиваются красным
6. Рецепционист открывает Drawer с деталями заявки
7. Нажимает Confirm → статус меняется на `confirmed`
8. Клиент получает email подтверждения, рецепция получает Telegram
9. Cron за 2 часа отправляет email-напоминание клиенту
10. Cron за 30 минут отправляет Telegram рецепции

5.2 Front-office — публикация профиля модели

1. Новая модель создаётся в admin (вручную или через Quick Upload)
2. При изменении статуса на `Active` вызывается `revalidatePath()`
3. Next.js ISR инвалидирует кэш страниц `/companions/[slug]` и `/companions`
4. При следующем запросе страница перегенерируется с актуальными данными
5. Профиль модели доступен на `/companions/{slug}` для клиентов
6. При открытии профиля `<ViewTracker>` инкрементирует viewsTotal

5.3 Процесс загрузки анкеты через Quick Upload

1. Пользователь нажимает Quick Upload в `/admin/models`
2. Открывается модальное окно или блок загрузки
3. Загружается PDF или фото анкеты
4. POST `/api/v1/models/quick-upload` с multipart/form-data
5. Файл передаётся в Claude с prompt на извлечение 35+ полей
6. Claude возвращает JSON с данными анкеты
7. Система парсит JSON (убирает markdown-обёртку при необходимости)
8. Создаётся запись Model в БД со всеми заполненными полями
9. Создаются ModelRate записи по найденным тарифам
10. Создаются ModelService записи по найденным сервисам
11. Фото загружаются в R2 с наложением водяного знака через Sharp
12. Обновляется coverPhotoUrl и galleryPhotoUrls в модели
13. Вызывается revalidatePath() для инвалидации кэша
14. Возвращается отчёт: заполнено N полей, пустых M, создано K сервисов, L тарифов

15. Пользователь перенаправляется на `/admin/models/{id}`

5.4 SEO-машина для районов Лондона

1. При build generateStaticParams() возвращает 25 slug-ов районов
2. Для каждого `[slug]-escorts` генерируется статическая страница
3. generateMetadata() загружает seoTitle/seoDescription из БД (или генерирует дефолтные)
4. На странице: модели в районе, transport hubs, hotels, restaurants, соседние районы, FAQ, Schema.org
5. ISR revalidate = 3600 — страница обновляется не чаще раза в час
6. При добавлении нового района — следующий build добавит его в sitemap и ISR

---

6. ПОТОК ДАННЫХ В СИСТЕМЕ

Поток 1: Добавление модели через Quick Upload

```
Анкета (PDF/фото)
    ↓
POST /api/v1/models/quick-upload
    ↓
Anthropic Claude API (извлечение 35+ полей)
    ↓
JSON с данными модели
    ↓
prisma.model.create() → PostgreSQL/Neon
    ↓
prisma.modelRate.createMany() (тарифы)
prisma.modelService.createMany() (сервисы)
    ↓
Sharp (наложение водяного знака VIREL)
    ↓
Cloudflare R2 (сохранение фото)
    ↓
prisma.model.update() (coverPhotoUrl, galleryPhotoUrls)
    ↓
revalidatePath('/companions')
revalidatePath('/companions/{slug}')
    ↓
Vercel ISR обновляет публичные страницы
    ↓
Модель видна в /companions на фронте
```

Поток 2: Цикл бронирования

```
Клиент → /book (3 шага)
    ↓
POST /api/public/booking-request
    ↓
prisma.bookingRequest.create() → PostgreSQL
    ↓
sendBookingReceived(clientEmail)  → Resend → Email клиенту
notifyReception(text)             → Telegram → Рецепция
    ↓
Рецепция → /admin/booking-requests → Confirm
    ↓
PATCH /api/v1/booking-requests/[id]/confirm
    ↓
prisma.bookingRequest.update(status: 'confirmed')
    ↓
sendBookingConfirmed(clientEmail) → Resend → Email клиенту
notifyReception(confirmText)      → Telegram → Рецепция
    ↓
Cron (*/15): проверка за 2 часа
    → sendBookingReminder(clientEmail) → Email напоминание
Cron (*/15): проверка за 30 минут
    → notifyReception(reminder30Text)  → Telegram рецепции
```

Поток 3: Просмотр профиля модели

```
Клиент → /companions/sofia-ldn
    ↓
Next.js ISR: проверка кэша
    ↓ (если кэш актуален)
Отдача статической страницы
    ↓
<ViewTracker> (Client Component)
    ↓
POST /api/public/models/sofia-ldn/view
    ↓
prisma.model.update(viewsTotal++, viewsToday++)
    ↓
Dashboard → Top Viewed Today (viewsToday)
```

Поток 4: SEO индексация

```
Vercel Build
    ↓
```

```
generateStaticParams() → 5 моделей + 25 районов + N сервисов
    ↓
generateMetadata() → title, description, canonical, OG
    ↓
Статические страницы с Schema.org JSON-LD
    ↓
/sitemap.xml → все URL с lastModified
    ↓
/robots.txt → Allow: / Disallow: /admin/, /api/
    ↓
Индексация поисковыми системами
```

---

7. ТЕКУЩИЙ СТАТУС СИСТЕМЫ

Полностью реализовано и работает

Компонент	Описание
Prisma Schema	12+ моделей, 40+ новых полей в Model
Seed данных	8 call rates, 98 сервисов, 25 районов, 20 хабов
Admin API	25+ endpoints с auth и audit logging
Public API	9 endpoints без auth
Admin UI	Dashboard, Models, Booking Requests, Masters, Blog, Concierge, Payments, Analytics
Model Form (Wizard)	4-шаговая форма с полными справочниками
Public Frontend	Все страницы: главная, каталог, профили, booking, smart match, services, districts
Auth система	Login/logout, UserRole RBAC, passwordHash
Cloudflare R2	Загрузка и хранение фото
Anthropic AI	Quick Upload, Generate Bio, Verify Photos, Generate SEO, Smart Match, Blog
Sentry	Мониторинг ошибок в production
Upstash Redis	Rate limiting на /api/auth/login
SEO система	Metadata, Schema.org, ISR, Sitemap, Robots

Компонент	Описание
Trust & Safety	Panic Button, Badges, Disclaimer, Age Check, GDPR Banner
Analytics	View tracking, Dashboard виджеты, Client CRM
Notifications	Инфраструктура полностью готова (graceful degradation)
Cron Jobs	14 jobs в vercel.json, защита CRON_SECRET
Footer	4 колонки с динамическими данными из БД

Готово к активации (токены не подключены)

Компонент	Необходимые ENV переменные
Email уведомления	RESEND_API_KEY, EMAIL_FROM
Telegram уведомления	TELEGRAM_DIVA_BOT_TOKEN, TELEGRAM_ADMIN_CHAT_ID
WhatsApp кнопки	AGENCY_WHATSAPP

Требует выполнения

Задача	Приоритет
Quick Upload: исправление полноты извлечения и синхронизации	Высокий
Шаг 10: Final QA + загрузка реальных моделей с фото	Высокий
Водяной знак VIREL: подключение Sharp в upload pipeline	Средний
Footer: деплой и проверка	Средний
Фаза 4: Members Portal (личный кабинет клиентов)	Будущий этап

---

8. ТЕХНИЧЕСКИЕ ЗАМЕТКИ

8.1 Архитектурные решения

Server Components vs Client Components. Страницы-оболочки реализованы как Server Components с загрузкой данных через Prisma. Интерактивные части (фильтры, формы, gallery) — Client Components. Данные передаются через props. Это обеспечивает SEO и производительность.

ISR (Incremental Static Regeneration). Все публичные страницы используют ISR вместо полного SSR. При изменении данных в admin вызывается `revalidatePath()` для точечной инвалидации кэша. Это исключает задержку до следующей ISR-ревалидации.

Flat API vs Grouped API. Первоначально API `/api/v1/services` возвращал сгруппированный формат `{ data: { categories } }`. Это было изменено на плоский массив `{ services, stats }` для совместимости со стандартными паттернами работы с данными в UI.

RBAC через Junction Table. Роли реализованы через отдельную таблицу UserRole (userId, roleId), а не через enum-поле в User. Это позволяет присваивать несколько ролей одному пользователю.

8.2 Ограничения текущей реализации

Notifications. Уведомления полностью реализованы на уровне кода. Для активации требуется добавить RESEND_API_KEY и TELEGRAM_DIVA_BOT_TOKEN в переменные окружения Vercel.

Watermark. Логика водяного знака написана, подключение в upload pipeline — предстоит.

Quick Upload. AI-извлечение работает, однако требует доработки prompt и маппинга полей для полного покрытия всех разделов анкеты.

Members Portal. Заглушка `/members` создана. Полная функциональность личного кабинета клиента — следующий этап разработки (Фаза 4).

8.3 Переменные окружения

Переменная	Статус	Назначение
DATABASE_URL	✅	Pooler URL Neon (eu-central-1)
DIRECT_URL	✅	Прямой URL Neon для миграций
NEXTAUTH_SECRET	✅	Подпись JWT токенов
NEXTAUTH_URL	✅	https://virel-v3.vercel.app
ANTHROPIC_API_KEY	✅	AI функции
R2_ACCOUNT_ID	✅	d53624886655cdd8ec0575389906dd2c
R2_BUCKET_NAME	✅	virel-operations-eu
R2_ACCESS_KEY_ID	✅	Cloudflare R2 токен
R2_SECRET_ACCESS_KEY	✅	Cloudflare R2 токен
CRON_SECRET	✅	Защита cron endpoints
SENTRY_DSN	✅	Мониторинг ошибок

Переменная	Статус	Назначение
UPSTASH_REDIS_REST_URL	✅	Rate limiting
UPSTASH_REDIS_REST_TOKEN	✅	Rate limiting
RESEND_API_KEY	❌	Email уведомления
EMAIL_FROM	❌	Адрес отправителя
TELEGRAM_DIVA_BOT_TOKEN	❌	Telegram Bot
TELEGRAM_ADMIN_CHAT_ID	❌	ID чата рецепции
AGENCY_WHATSAPP	❌	Номер для wa.me ссылок
AGENCY_TELEGRAM	❌	Username для t.me ссылок

8.4 Важные URL и идентификаторы

Ресурс	Значение
Production URL	https://virel-v3.vercel.app
GitHub	github.com/adelislamov-star/virel-v3
Vercel Project ID	prj_bOjDo27XJ0kgKFxsbKXS2paWC6f5
Vercel Team ID	team_59P1Zdn0cguulnBaagCpyIa7
R2 CDN	https://pub-2df895cf47a14d359a7341e0cda3efaf.r2.dev
R2 Bucket	virel-operations-eu
DB Region	Neon EU Central (eu-central-1)

8.5 Структура директорий

```
src/
├── app/
│   ├── (public)/          — Front-office страницы
│   ├── admin/             — Back-office страницы
│   ├── api/
│   │   ├── v1/            — Защищённые admin API endpoints
│   │   ├── public/        — Публичные API endpoints
│   │   └── cron/          — Cron job handlers
│   ├── sitemap.ts
│   └── robots.ts
```

```
├── components/
│   ├── public/            — Компоненты публичного фронта
│   └── admin/             — Компоненты admin панели
├── constants/
│   └── model-form.ts      — Справочники: станции, национальности, языки
├── lib/
│   ├── prisma.ts          — Prisma client
│   ├── email.ts           — Email функции (Resend)
│   ├── telegram.ts        — Telegram функции
│   ├── telegram-messages.ts — Шаблоны Telegram сообщений
│   ├── auth.ts            — Аутентификация
│   └── watermark.ts       — Sharp водяной знак
├── types/                 — TypeScript типы
└── prisma/
    └── schema.prisma      — Схема базы данных
```

Документ составлен: 14 марта 2026
Проект: Virel v3 | Владелец: Adel Islamov | AI-ассистент: Claude (Anthropic)